Language Architecture & Notation Grouping (LANG)

More code, more people, more developers, more data and all the code generated instead of written continues making the system of many languages across many people less relevant.

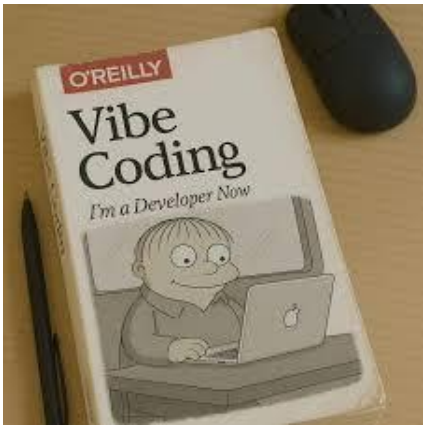
The Problem

Software development has never involved more people, more deployment targets, or more languages simultaneously in active use than it does right now.

More People

Programming has been expanding its audience for decades without stopping. It began with researchers and engineers in institutions, moved to computer science graduates, then to self-taught developers, and more recently to an explosion driven by open source, coding bootcamps, and platforms that make code publicly accessible and shareable. Today, people with no formal technical background are building working software through agentic coding tools — projects that would have required experienced engineers five years ago.

LinkedIn's 2024 data listed Full Stack Developer as one of the ten most in-demand jobs, with year-over-year postings increasing over 35%. The U.S. Bureau of Labor Statistics projects 16% growth in web developer roles through 2032. Full-stack development — where a single developer or small team owns the entire stack from systems to web — has become the dominant professional norm rather than the exception it once was. For most of computing history, frontend, backend, and systems were treated as separate disciplines requiring separate specialists. That has changed.



More Languages

The three domains of software development each have a dominant language that is not going away. C owns systems, embedded, automotive, and firmware — anything where runtime overhead is constrained and timing guarantees are required. Python owns scientific computing, data engineering, machine learning infrastructure, and scripted automation. JavaScript and HTML own the web, front and back. These are not interchangeable. Each has a different memory model, a different runtime, a different type system, and different conventions for expressing the same underlying logic.

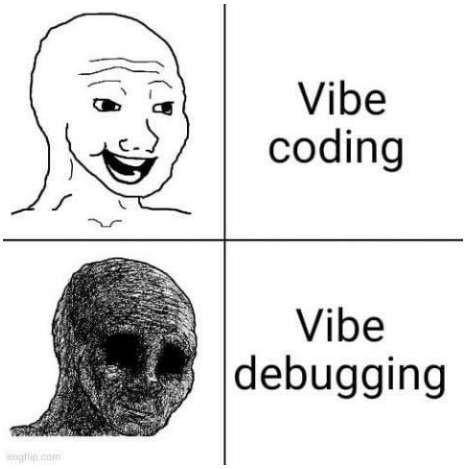
A developer working across all three operates in three separate syntax systems, three separate toolchains, and three separate mental models. The TIOBE index has documented that Python has become the only language in decades to surpass C in measured usage — yet C remains in the top two, driven by the enormous volume of embedded, automotive, and infrastructure code that will run

for decades. Both languages are simultaneously in heavy use, by different populations, with no common ground between them.

Coding Agents and the Output Problem

Agentic coding tools have reached a point where generating code from a natural language description is possible at useful quality. What remains unsolved is what happens after the output is produced — whether the person receiving it can read it, verify it, modify it, or diagnose it when something goes wrong.

Agents currently emit into whatever language they infer from context. The selection is implicit, inconsistent across sessions, and produces output in a language the receiver may not know well enough to evaluate. A hobbyist who prompted their way to a working prototype sees hundreds of lines of code they cannot read when something breaks. The same prompt run twice produces different library choices, different error handling patterns, and different structural decisions — all working, none the same. A codebase built incrementally through agent sessions looks like it was written by multiple developers with different preferences and no coordination.



Why Unification Has Not Happened

The idea of a unified programming language is not new. IBM's PL/I, introduced in 1964, was designed explicitly to unify scientific, business, and systems programming under a single syntax. It failed because it attempted unification by inclusion — taking features from every domain and combining them into one language. The result was large enough that nobody achieved mastery of it, and the populations it was trying to serve stayed in their existing languages because the migration cost exceeded the benefit.

Every subsequent attempt reproduced one or both of its failure modes. Languages that tried to do everything became unwieldy. Languages that targeted a specific domain well faced the ecosystem problem: tooling, packages, and community built around established languages represent a compounding investment that a new entrant cannot replicate quickly. There is also a third failure mode that receives less attention: the problem that unified languages were designed to solve was not widely felt at the time most of them were proposed. For most of the field's history, developers specialized. The full-stack developer spanning systems, application, and web was rare. The conditions that would have made a unified representation valuable earlier are present now in a way they were not before.

UniLogic (UL)

UL is a unified programming language that transpiles to C, Python, and JavaScript/HTML. One syntax covers the range from memory-level systems work to web application development. The design principle is minimization without loss of power. The keyword set covers only what is genuinely distinct across the three target languages. Everything composable from existing keywords is not a keyword. What falls outside the defined set entirely is expressed as an inline native block — the developer needs to understand only that specific construct, not the full language it comes from. The result is a syntax set small enough to be learned once, powerful enough to cover what C, Python, and JavaScript collectively cover, and consistent enough that code written by ten different developers using it looks like it was written by one.

Syntax

UL is explicitly end-terminated. Every block opens with a keyword and closes with end keyword. No braces. No significant whitespace. One statement per line.

Variables, types, control flow

```
int x = 10
float pi = 3.14159
string msg = "hello"
bool active = true

if score > 200
  print "high score"
else if score > 100
  print "good score"
else
  print "keep trying"
end if

for int i = 0 while i < 10 do i++
  print i
end for

match code
  1: print "one"
  2: print "two"
  default: print "other"
end match
```

Functions and types

```
function add(int a, int b) returns int
  return a + b
end function

type Player
  string name
  int score = 0
end type

Player p = new Player["Alex", 98]
print p.name
```

Memory operations

The angle bracket is the consistent signal that memory operations are in play. Nothing else in the syntax uses angle brackets.

C	UniLogic
`int x = 42;` `int *p = &x;`	`int x = 42` `int <p> = address(x)`
`int copy = *p;` `*p = 100;`	`int copy = deref(<p>)` `deref(<p>) = 100`
`int *q = NULL;` `int **pp = &p;`	`int <q> = empty` `int <<pp>> = address(<p>)`
`int *arr = malloc(sizeof(int)*10);` `arr[0] = 1;`	`int <arr> = alloc(size(int) * 10)` `deref(<arr>) = 1`
`free(arr);` `int (*fn)(int,int) = &add;`	`free(<arr>)` `fn<int,int> = address(add)`
`int r = fn(3, 4);`	`int r = call(<fn>, 3, 4)`
`Player *pp2 = &p;` `pp2->score = 100;`	`int <pp2> = address(p)` `deref(<pp2>).score = 100`
`void *vp = (void*)arr;` `int *ip = (int*)vp;`	`none <vp> = change(<arr>)->none` `int <ip> = change(<vp>)->int`
`*(uint32_t*)0xDEAD0000 = 1;`	`mem(<0xDEAD0000>) = 1`

Error handling — Result and Option

UL does not have exception syntax. Functions that can fail return Result. Functions that may return nothing return Option. The type system enforces handling — the compiler rejects code that ignores the error case. The ? operator propagates errors up the call stack without boilerplate.

```
function read_file(string path) returns Result(string, IOError)
  // ...
end function

match read_file("data.txt")
  ok(contents) -> print contents
  err(e)       -> print "failed: " + e.message
end match

// ? propagates err immediately, unwraps ok and continues
function process(string path) returns Result(int, IOError)
  string contents = read_file(path)?
  return ok(contents.length())
end function
```

Inline native blocks

Any construct outside the defined keyword set is expressed as an inline native block. The surrounding UL code is unchanged. An IDE surfaces an explanation of what the block does — the developer learns only that specific construct, not the full language.

```
function get_thread_id() returns int
  inline c {
    return (int)pthread_self();
  }
end function
```

Cross-Language Work and XPile

UL supports bidirectional transpilation. A developer can take code written in Python, C, or JavaScript, transpile it to UL, make edits, and transpile back to the original language. The team maintaining the original codebase sees a diff in their own language. The editing developer never had to learn it.

This addresses a specific and common problem: two teams that interact through code in different languages. Currently, modifying the other team's code requires either learning their language or asking them to make the change — both slow options. With bidirectional transpilation, the editing developer can hand back a working implementation rather than a written description of what they want. The feedback loop collapses from multiple clarification cycles to a single review.

XPile handles language-to-UL transpilation, built one source language at a time. Python, C, and JavaScript/HTML are supported bidirectionally as part of the core system. Other languages — Java, Go, and others — are added incrementally by demand. Once several languages are supported, XPile accepts a source file, an output path, and a language flag, and produces UL regardless of the source.

Dynamic Runtime (DR)

Every programming language makes founding decisions about runtime behavior — how memory is managed, whether types are checked at compile time or runtime, how wide integers are, whether safety checks are enforced. These decisions are made once by the language designer and locked in permanently. The programmer inherits whatever the language chose.

UL has no founding runtime decisions. Runtime behavior is declared by the developer in a DR block at the top of the program. Nine settings are available. The compiler enforces whatever is declared uniformly across the standard library — not just in application code, but in every function, data

structure, and IO operation the standard library provides. Not an escape hatch from a default, but a genuine declaration surface where no default exists.

```
runtime {
  memory:      gc          // gc | manual | refcount
  safety:      checked     // checked | unchecked
  int_width:   32          // 32 | 64 | platform
  types:       strict      // strict | dynamic
  concurrency: threads     // threads | async
  // ... 4 further settings
}
```

Setting	What it controls
memory	gc for automatic collection, manual for explicit alloc/free, refcount for reference-counted lifetimes. Arena allocation is a separate stdlib type — an allocator object the developer creates, passes around, and frees as a unit.
safety	checked enforces bounds checking, null safety, and integer overflow detection. unchecked removes these for performance-sensitive paths.
int_width	32 or 64 bit default integer width, or platform to match the target architecture's native width.
types	strict enforces static typing at compile time. dynamic allows runtime type inference. strict is required when compiling to C.
concurrency	threads emits OS-scheduled threading. async emits event-loop non-blocking code — mapped to JS Promises, Python asyncio, or JVM virtual threads depending on target.

A program targeting embedded firmware declares memory: manual, safety: unchecked, int_width: platform. A data pipeline declares memory: gc, types: dynamic. The syntax is the same. The compiler generates fundamentally different output. Compatibility between settings is verified at compile time — some combinations are hard errors, others produce warnings, the rest are silent.

@Mods — Compiler Modifiers

Beyond the DR settings, UL provides compiler modifiers prefixed with @ that extend the behavior of declarations without adding keywords to the core syntax. Placed on the line before a declaration, they cover concurrency, platform-specific targeting, safety annotations, type behavior, and function-level optimizations.

Modifier	What it does
@async / @await	Marks a function as a coroutine. Maps to JS Promises, Python asyncio, or JVM virtual threads depending on compile target.
@platform(c)	Restricts a function to a specific compile target. Multiple platform overrides for the same function name let the compiler select the correct implementation automatically.
@nullable	Marks a field or parameter as explicitly nullable. Without this modifier the compiler assumes non-null and enforces it.
@inline	Substitutes the function body at every call site at compile time. Enforced — the compiler will not ignore it.
@operator	Maps a named function to an operator symbol for a type. Enables clean arithmetic on custom types without special syntax.
@delegate	Forwards all interface calls on a type to a named field of that interface type. The compiler generates every forwarding method.

"Our no-code solution lets you easily create data pipelines from a config"

The config:

```
{
  "Workflow": "UserTransactionsPipeline",
  "Instructions": [
    "import pandas as pd",
    "df = pd.read_csv('USER_TRANSACTIONS_SNAPSHOT.csv')",
    "df.dropna(subset=['id_deprecated', 'column9'], inplace=True)",
    "df['product_code'] = df['product'].apply(lambda x: x.strip().lower
    ())",
    "df = df[df['amount'] > 0]",
    "df['occurred_at'] = pd.to_datetime(df['occurred_at'],
    errors='coerce')",
    "df.to_sql('user_transactions', con=engine, if_exists='append')"
  ]
}
```

@Brooklyn Anticapital
Habitat Defense Systems

Pragma Language (PL)

Pragma is a backend data movement layer that sits above UL. It is not a replacement for programming — it is a schema language for stateless sequential pipelines, programs where data flows through a fixed sequence of operations with no continuous state management and no complex branching that depends on runtime conditions.

The population that builds these programs is large and growing. Fetch data from an API, transform it, save it somewhere, notify someone — this pattern repeats across automation scripts, data pipelines, scheduled jobs, webhook handlers, ETL processes, and integration glue between services. It currently requires 50 to 100 lines of boilerplate code every time, written slightly differently by every developer who writes it, with the meaningful logic buried in scaffolding that varies by author and session.

Pragma replaces that scaffolding with a declaration. The action is the header. The things involved and any variation in what or how are expressed as key-value attribute pairs beneath it. The output of each operation is assigned a name so subsequent operations can reference it. The compiler generates all scaffolding from the declaration. The developer specifies only what varies.

Syntax

Simple pipeline

```
fetch:
    source=api.example.com
    method=get
    auth=myauth
    name=result1

convert:
    content=result1
    from=json
    to=xlsx
    name=converted1

save:
    content=converted1
```

```
    dest=C:/output/results.xlsx
```

Fallback and error handling

Any attribute that can fail carries its own fallback inline using 'or' inside brackets. A comma inside brackets means multiple accepted values simultaneously. 'or' means ordered fallback — try left first, then right.

```
fetch:
    source=[api.primary.com or api.backup.com]
    auth=[key1 or key2]
    format=[json,csv]
    name=result1
```

Scheduling and notification

```
schedule:
    action=dailypipeline
    every=1d
    on_fail=notify

notify:
    via=[slack or email]
    dest=[#alerts or admin@example.com]
    content="Pipeline complete"
```

Arithmetic on named values

```
save:
    name=dollarvalue
    content=[btcamount * btcprice]

save:
    name=lambocount
    content=[dollarvalue / 450000]

output:
    content=lambocount
    to=ui_display
```

The Ten Actions

Action	What it does
fetch	Pull data from an HTTP endpoint, file, database, or hardware device
save	Write data to a destination or assign a computed value to a named reference
delete	Remove a record, file, or object from a destination
convert	Transform data from one format to another, or apply a processing method such as OCR
notify	Send a notification via email, SMS, webhook, Slack, Discord, or popup
schedule	Run a named action or sequence on a recurring basis or at a specified time
input	Define a named value that the user or calling system provides
output	Send a named value to a destination — console, log, or a named UI element
auth	Define a named authentication configuration referenced by other actions
query	Retrieve a filtered subset of data from a database

UL Components as Named References

Any UL component — an auth flow, a transform algorithm, a device driver — can be referenced in Pragma by name as an attribute value. The complexity lives in the UL file. Pragma sees only a name. The keyword count stays fixed. Capability expands through the library.

```
auth:
    method=ourauth1
    credentials=secrets.conf
    name=session1

fetch:
    source=api.example.com
    auth=session1
    name=result1
```

Bidirectional Compilation

Pragma generates UL, which compiles to the target language. When UL is edited directly, the compiler folds changes back into Pragma. Where a UL construct has no Pragma representation, the compiler surfaces it interactively and asks the developer how they want to handle it: give it a name and reference it in Pragma, simplify the UL so it is expressible, or leave it as UL-only. The Pragma file stays accurate as a high-level view of the pipeline as the codebase evolves. Changing an endpoint, swapping a processing package, adding a notification step, reordering operations — these are single attribute edits on single lines in the Pragma file, propagated down to the generated code automatically.

What Pragma Covers and What It Does Not

Pragma covers backend data operations: HTTP requests, file reads and writes, database queries, data transformation, format conversion, scheduling, notifications, authentication, and device communication where the interface is a read or write operation. It covers simple arithmetic on named values. It does not cover UI rendering, continuous state management, or procedural logic. The natural test: can the outcome be fully specified by declaring what you want, without describing how to achieve it? If yes, it is a valid Pragma construct. If achieving the outcome requires procedural logic that varies by context, it belongs in UL.